

eventus: Object-Oriented Longitudinal Cohort Analysis with Episodes, Events, and Observation Periods

Daniel J. Arenas,¹ Riccardo Fincato²

2026-06-12

Abstract

Longitudinal cohort analysis — characterizing how entities behave within defined observation periods through interval episodes and point-in-time events — is a recurring challenge in multiple fields. Tools exist for parts of this problem — schema validation, cohort definition, survival and adherence analytics, and pipeline configuration — but, to our knowledge, no existing Python or R package combines them into a unified framework in which observation periods, interval episodes, and point-in-time events are treated as first-class typed objects with validated construction, declared and versioned decisions, and explicit relationships between them. Analysts typically must write bespoke scripts that couple data cleaning, analytical computation, and visualization in ways that are difficult to maintain, audit, or reproduce across datasets. We present **eventus**, a domain-agnostic Python package that structures this class of analysis through object-oriented abstractions: semantics, data objects, cleaners, analyzers, intermediates, and visualizers. Each class has a single responsibility and trusts the objects it receives from other classes. Cleaning decisions are versioned in YAML configuration files and produce per-row audit trails. Analytical results are typed intermediate objects that can easily carry additional information such as validated denominators and explicit eligibility thresholds. Visualization decisions live in versioned configuration files independent of analysis code. We demonstrate the package through ten vignettes on simulated data covering insurance coverage periods, nursing facility stays, emergency department visits, inpatient hospitalizations, and event co-occurrence analysis. **eventus** is available at <https://github.com/djarenas/eventus> (currently private prior to publication).

Keywords: longitudinal data analysis, cohort analysis, health services research, reproducibility, object-oriented programming, Python, episodes, events, observation periods

Introduction

The ubiquity and unreliability of longitudinal data cleaning and analysis

A recurring challenge in applied data science, particularly in health services research, insurance analytics, and longitudinal program evaluation, is the need to characterize interval episodes and point-in-time events within defined periods of time. A cohort may contain patients observed over several years, with intermittent insurance coverage, who make discrete visits to the emergency department and experience inpatient

¹ Affiliation, address, and email to be added.

² Affiliation, address, and email to be added.

hospitalizations. Before any scientific question can be asked, a seemingly mundane one must be answered: *“Do I know what I have?”*

Data cleaning in longitudinal studies is ubiquitous, consequential, and easy to under-document. Hospitalization records may have missing discharge dates or ones that precede admission dates. Emergency department claims may contain null patient identifiers. Coverage files may contain exact duplicate rows, overlapping intervals that represent transfers rather than re-admissions, and records that extend beyond the study window. Records may also require merging: ED visits on the same day may represent one visit billed twice, or may represent two genuine visits to different hospitals: a distinction that depends on a declared analytical rule, not a default assumption.

The analyst who does not handle these issues systematically introduces silent errors that propagate through every downstream calculation. The analyst who handles them in a script carries the risk of handling them invisibly: the decisions made on line 12 of a cleaning function are inaccessible to any collaborator who reads only the output, unreproducible without reading the exact code, and undocumented by default. Many published analyses in health services research provide limited documentation of data cleaning decisions: how duplicates were handled, how causality violations were resolved, or how many records were excluded and why. These practices contribute to the irreproducibility of data analysis and modeling. Broader analyses of computational reproducibility suggest this is not unique to data cleaning and not unique to health services research (Peng 2011; Hemkens et al. 2016). Journal reproducibility policies have been shown to be a good step in the right direction but still ineffective at ensuring that published analyses can be reconstructed from reported methods (Stodden, Seiler, and Ma 2018).

Careful documentation of all analytical steps has been widely advocated (Sandve et al. 2013; Wilson et al. 2017). In our experience reviewing and conducting longitudinal analyses in health services research, cleaning, analysis, and visualization decisions are not always reported in sufficient detail to reproduce the pipeline from the methods section alone. Scripts do not produce audit trails, and collaborators and reviewers cannot be expected to carefully sift through massive scripts implementing various cleaning choices and their consequences. This is not negligence; it is a structural limitation of the script-based paradigm. In contrast, configuration-driven software with declarative construction, such as **eventus**, aims to produce easily auditable cleaning choices and consequences.

A second and underappreciated consequence of the config-driven design is that it facilitates systematic examination of how conclusions vary across reasonable analytical choices. Any longitudinal analysis involves decisions that reasonable researchers would make differently — what gap length constitutes a coverage lapse, whether causality violations should be rejected or repaired, how many days before a hospitalization an emergency department visit counts as clinically related. These are not errors. They are legitimate choices with no single correct answer, and the conclusions of an analysis may depend on them. Several frameworks exist for systematically examining variations in methodological choices: multiverse analysis, which examines how results vary across the space of reasonable analytical choices (Steege et al. 2016); vibration of effects for assessing how associations shift across model specifications (Patel, Burford, and Ioannidis 2015); and

specification curve analysis for visualizing results across all defensible model specifications (Simonsohn, Simmons, and Nelson 2020). All share an important core: run the analysis across a range of reasonable analytical decision rules and report how the conclusions vary. This extends classical sensitivity analysis, which varies data inputs while holding the analysis code fixed. In a script-based pipeline, examining various sensitivity and/or multiverse-analysis choices may require maintaining multiple versions of the same code and manually tracking which version produced which output — a practical barrier that many analysts may not overcome. In the **eventus** design, it requires writing multiple YAML configuration files (hereafter: config files) and running the same pipeline once per file. An analyst studying coverage gaps who wants to know whether their findings are robust to a lapse definition of 7, 14, or 30 days runs three config files. The results are directly comparable because the script does not have to be changed at multiple points. YAML files may also be easier to document, share with collaborators, and attach to a journal submission than three versions of the same script.

Although motivated by health services research, the structure that **eventus** addresses — entities, observation periods, interval episodes, and point-in-time events — recurs across domains wherever entities are observed over time: insurance analytics, educational longitudinal studies, ecological monitoring, clinical trial follow-up, and workforce analysis. The framework is domain-agnostic by design. All classes accept user-defined column names through semantics objects. This paper uses simulated data on insurance coverage periods, nursing facility stays, emergency department visits, and inpatient hospitalizations as its running example, but the design principles apply wherever the three-object structure appears. We hope this work is useful not only to analysts in health services research but also to researchers in other domains and to software developers designing similar frameworks.

The script-based paradigm and its structural limitations

The dominant paradigm for longitudinal cohort analysis is the analysis script: a sequence of `DataFrame` transformations, hardcoded column references, and inline plotting code. This approach is fast to write and easy to follow when the analysis is self-contained. But it becomes a liability when the analysis must be reproduced, shared, extended, or audited. A script that cleans hospitalization records on line 12, filters to an observation period on line 47, and computes coverage statistics on line 83 embeds three distinct scientific decisions (what counts as a duplicate, what counts as in-window, what counts as covered) in ways that are invisible to any collaborator who reads only the output.

Beyond the cleaning problems described above, the script-based paradigm offers no mechanism for carrying guarantees across analytical steps. A function that receives a `DataFrame` cannot know whether that `DataFrame` is clean, whether its columns mean what they appear to mean, or whether the decisions that produced it were deliberate or accidental. Rename a column in the data warehouse and the script may break silently in multiple places. Change an aggregation rule and the change is not recorded anywhere. The denominator of a percentage statistic is tracked manually — one misplaced filter produces wrong statistics silently, with no error and no warning. A script writer who wants data safety must write defensive validation functions manually, remember to call them, pass the

right DataFrames to them, and update them when the schema changes. Every guarantee is opt-in and fragile.

Object-oriented programming offers a structural alternative to this fragility. Data objects can enforce their own validity at construction, so that a valid object is a guarantee rather than a hope (Martin 2000). The **eventus** design extends **pandas** by wrapping DataFrames in typed objects that carry domain semantics: what the columns mean, what constraints must hold, and what decisions produced the data. The DataFrame becomes an attribute of the object rather than the object itself.

The generality of the underlying tools is part of why this fragility persists. **pandas** (McKinney 2010) is powerful precisely because of its agnosticism. It does not know that `discharge_date` should follow `admit_date`, that `patient_id` should never be null, or that two records with the same entity, date, and hospital represent a duplicate while two records at different hospitals do not. This agnosticism is appropriate for a general-purpose library; however, it means that every application built on **pandas** must either enforce these rules itself or silently accept violations. The same is true of the general-purpose data-manipulation and date-handling tools that longitudinal analyses lean on, such as **dplyr** (Wickham et al. 2023) and **lubridate** (Grolemund and Wickham 2011): they operate on raw DataFrames and carry no mechanism for guaranteeing structural soundness across steps. Section 1.4 situates **eventus** against the more specialized tools — data validation frameworks, cohort phenotyping systems, temporal analytics packages, and pipeline managers — that each address part of this problem.

eventus: Object-oriented longitudinal cohort analysis

eventus addresses this gap by treating longitudinal cohort analysis as a software engineering problem rather than a scripting one. It does not replace **pandas** — it elevates it by wrapping DataFrames in typed objects that carry domain semantics, structural guarantees, and declared decisions. The framework rests on seven architectural commitments.

Validated objects as guarantees. If a data object exists, it is complete. Constructors raise on invalid inputs, so every function or class that receives a data object can trust what it holds. An `Episodes` object is not a DataFrame with hospitalization data; it is a guarantee that every row has a valid entity identifier, a start date, an end date, and that start precedes or equals end. A visualizer that receives an `Episodes` object does not check whether the data is valid or whether the columns exist — it checks only that it has an `Episodes` object. The guarantee travels with the data.

Single responsibility. Each class is meant to represent one concept and do exactly one job. A cleaner cleans — it does not analyze. A data object holds validated data — it does not clean or plot. An analyzer computes — it does not re-validate inputs that upstream objects already guarantee. A visualizer renders — it does not recompute what an intermediate already carries. No class absorbs a neighboring concern for convenience, and no step skips a layer. This discipline is what makes the pipeline composable: each component can be replaced, extended, or tested independently because its responsibilities are bounded.

Explicit audit trails. Nothing should be silent. Cleaners produce structured audit reports documenting every transformation: how many episodes were merged, what overlaps were resolved, which duplicates were dropped. NaN values in intermediate objects are scientifically meaningful absences of signal, not missing data — an entity with fewer than two events genuinely has no mean inter-event gap, and the result object carries that honestly. No transformation is undocumented. No absence is hidden.

Declared decisions. Every cleaning, analytical, and visual decision is specified through configuration objects that can be built from versioned YAML files. Nothing is embedded in code. What counts as a duplicate, what gap length constitutes a coverage lapse, how descriptor columns are aggregated during episode merging — all are declared explicitly, recorded, and reproducible. Changing a decision means changing a config file, not hunting through code.

Schema decoupling. Column names are declared once in a semantics object and never referenced again downstream. Renaming a column in the data warehouse requires changing one line. The rest of the pipeline is unaffected because it never knew the column name — it knew the semantic role.

Typed intermediates as contracts. Analyzers and visualizers do not share raw DataFrames — they share typed intermediate objects. EventResultVolume carries per-entity event counts with validated denominators. EventCoOccurrenceGapTest carries a permutation null, observed gaps, and a KS statistic with explicit NaN semantics. A visualizer that receives EventCoOccurrenceGapTest knows exactly what it has. The column-naming problem — `n_cirrh_and_ed_visits_within_30d_of_dx` — disappears when results carry their own context. Intermediates close the gap between analyzers and visualizers with the same discipline as everything else: they carry exactly what is needed and nothing more.

The open/closed principle. The architecture is designed for extension without modification. Adding the three co-occurrence analyzers — EventCoOccurrenceAnalyzer, EventCoOccurrenceGapAnalyzer, and EventCoOccurrenceDirectionalityAnalyzer — required writing three new classes that consumed existing validated objects. The CohortTimeline, the cleaners, and all prior analytical components required zero changes. New analytical capabilities are added by writing new components, not by modifying existing ones (Gamma et al. 1994; Martin 2000).

Beyond reproducibility, these commitments produce a shared vocabulary. When a team uses **eventus**, concepts such as ObsPeriodPerEntity, EpisodeSemantics, Episodes, and CohortTimeline acquire precise, shared meanings. Collaborators can discuss “the observation period for this cohort” or “the episode semantics for inpatient stays” and mean something unambiguous — not “the date range I filtered to on line 47” or “the cleaning I did last Tuesday.” Typed objects are not just engineering artifacts. They are conceptual tools that clarify scientific communication.

Related software

The seven commitments above are individually unremarkable — each is standard practice in some corner of the software ecosystem. What is missing is a tool that combines them for

longitudinal cohort analysis. It is therefore worth being precise about which neighboring tools supply which ingredient, and where each stops short. We group the most closely related software into four categories. None is a competitor to **eventus** in the sense of solving the same problem; each is better understood as adjacent, and in several cases complementary.

Data validation frameworks. A family of tools lets analysts declare a schema and validate a DataFrame against it at runtime: in Python, **pandera** (Bantilan 2020) and Great Expectations (Great Expectations Team 2024); in R, **pointblank** (Iannone and Vargas 2024) and **validate** (Loo and Jonge 2021). These are mature, carefully engineered tools, and they share **eventus**'s conviction that a validated object is worth more than an unvalidated one; **pandera**'s schema interface — returning the data if valid and raising otherwise — is close in spirit to **eventus**'s constructors, and we drew encouragement from seeing the idea so cleanly realized. The difference is one of scope, and it is a deliberate one on their part. Because these frameworks are domain-agnostic — designed to validate any tabular data, whether it describes patients, financial transactions, sensor readings, or anything else — their schema vocabulary is necessarily general: column types, value ranges, and user-supplied row- or column-level checks. That generality is exactly what makes them broadly useful, but it also means a **pandera** schema cannot express that an overlap between two hospitalization intervals at the same facility is a merge candidate while an overlap at different facilities is two distinct episodes, because that is not a property of a single row or column — it is a declared decision about interval structure, specific to the episode/event domain. Validation frameworks also stop at the validation boundary by design: they certify that data conforms to a schema, then hand the raw DataFrame onward; they do not carry the guarantee into analysis or visualization, nor produce a per-row audit trail of cleaning transformations. **eventus** can be seen as taking the validated-schema idea these tools pioneered and specializing it — from domain-agnostic type and range checks to temporal, interval, and event semantics — then carrying those guarantees through the whole pipeline.

Cohort definition and phenotyping. In observational health research, the Observational Health Data Sciences and Informatics (OHDSI) collaborative (Hripcsak et al. 2015) provides a mature stack for defining cohorts against the OMOP Common Data Model — the product of a large international community effort sustained over more than a decade. Tools such as **Capr** (Lavalley and Black 2024) express cohort logic — entry events, inclusion criteria, exit strategies — in R that compiles to standardized SQL, and OpenSAFELY's ehrQL (Nab et al. 2024) plays an analogous role for NHS electronic health records. These systems are powerful and widely used, but they sit *upstream* of the problem **eventus** addresses. They answer the question “which persons, observed when, belong in my cohort?” and require the source data to be mapped to a common data model first. Their downstream analytics are organized around specific study designs — population-level effect estimation, patient-level prediction. **eventus** begins where a cohort already exists: it takes arbitrary tabular episode and event data, requires no common data model, and is agnostic to the analytical question. The two are naturally complementary — an OHDSI-defined cohort exported to DataFrames is exactly the kind of input **eventus** is built to clean, structure, and analyze.

Temporal and survival analytics. Several packages provide rigorous, often typed treatment of time-to-event and episode data within a specific methodological frame. The R **survival** package (Therneau 2023) represents censored time-to-event data in a dedicated `Surv` object — itself a validated typed object — and **AdhereR** (Dima and Dediu 2017) computes medication-adherence measures over observation windows from interval supply data, with an explicit emphasis on transparency and reproducibility that is close in motivation to our own. These tools are deep within their domains, but each is organized around one model family or one class of measure, generally assumes the input data is already clean, and provides neither config-driven cleaning with audit trails nor a shared intermediate that flows uniformly into multiple downstream analyzers and visualizers. **eventus** is broader and shallower by design: it does not implement survival models or adherence measures, but it supplies the validated, audited, semantically typed substrate on which such analyzers can be written as interchangeable components.

Pipeline and configuration frameworks. A final category supplies the reproducibility machinery — declarative configuration, dependency tracking, versioned execution — without committing to any analytical content. Kedro (Alam et al. 2024) organizes Python pipelines around typed data catalogs declared in YAML; Snakemake (Köster and Rahmann 2012) and the R **targets** package (Landau 2021) track dependencies and re-run only what changed. These are excellent, battle-tested systems, and **eventus** shares their commitment to declared, versioned configuration. We emphasize that they are not alternatives to **eventus** but potential hosts for it: each is a general-purpose orchestrator that runs *whatever* analysis the user writes, and an **eventus** pipeline can sit as a stage inside a Kedro, Snakemake, or **targets** workflow. The complementarity is clean precisely because the two operate at different layers. An orchestrator schedules and caches steps but is deliberately agnostic to what the steps mean: a Kedro catalog can version *which* cleaning function ran on *which* dataset, but has no concept of a duplicate hospitalization, a coverage gap, or an event-to-episode interval. **eventus** is the domain layer that supplies exactly those concepts — the validated objects, the semantics, the analyzers — and it is this layer, not the orchestration around it, that is designed to be extended: adding a new analytical capability means writing a new component against the existing typed objects, as the co-occurrence analyzers in Section 2.4 demonstrate. An orchestrator and **eventus** together give an analyst both halves: reproducible execution from the former, and an extensible vocabulary of validated longitudinal objects from the latter.

Across these four categories, each system supplies one ingredient of the **eventus** design — typed validation, cohort definition, temporal analytics, or declared configuration — and several are natural collaborators rather than alternatives. Building **eventus** gave us considerable respect for the authors of these tools; the apparent simplicity of a clean validation API or a readable configuration language conceals a great deal of careful design, and our framework stands on the conceptual groundwork they laid. To our knowledge, no existing Python or R package combines these ingredients into a single framework in which observation periods, interval episodes, and point-in-time events are validated typed objects whose cleaning, analytical, and visual decisions are declared, versioned, and audited end-to-end. And the combination is more than a bundling: the typed objects are designed to compose under defined rules. A validated `ObsPeriodPerEntity`, a validated

Episodes object, and a validated Events object are not merely co-resident in the same pipeline — they can be assembled and analyzed together through a small algebra of pairwise interactions ($\text{Events} \times \text{ObsPeriod}$, $\text{Episodes} \times \text{ObsPeriod}$, $\text{Events} \times \text{Events}$, $\text{Events} \times \text{Episodes}$), each yielding a typed result with validated denominators and explicit absence semantics. Because each operand has earned its structural guarantees upstream, these combinations compose safely without re-validation, and new interaction types can be added as new components rather than rewrites. It is this combination — end-to-end declared and audited guarantees, plus a composable algebra over the validated objects — rather than any single one of its parts, that is the contribution of **eventus**.

Paper organization and documentation

The remainder of this paper is organized as follows. Section 2 describes the software architecture and its core abstractions with emphasis on design decisions. Section 3 presents ten vignettes demonstrating the full pipeline on simulated insurance and clinical data. Section 4 compares **eventus** with equivalent script-based implementations. Section 5 describes additional analytical capabilities not covered in the vignettes. Section 6 discusses implications for reproducibility, generalizability, and future directions.

The package is organized into subfolders corresponding to each abstraction: semantics, data objects, cleaners, analyzers, intermediates, and visualizers. Each subfolder has a dedicated README providing full API reference, design notes, and usage examples. Computational workhorse code is separated into `_utils.py` modules, keeping the primary classes readable and focused on their single responsibility. The module READMEs are intended to serve as both human-readable documentation and machine-readable specifications for LLM-assisted development.

Software architecture

eventus is organized into six abstraction layers: semantics, data objects, cleaners, analyzers, intermediates, and visualizers. Each class within each layer has a single responsibility. Each component trusts the typed objects it receives rather than re-validating them — those guarantees were established upstream and are carried by the objects themselves. An analyzer that accepts a validated Episodes object does not need to verify that it has a start column, an end column, or that causality holds. A visualizer that receives a typed intermediate does not recompute what the intermediate already carries. Components can be composed flexibly: an intermediate produced by one analyzer can be passed to another analyzer before reaching a visualizer, and the CohortTimeline can be progressively enriched across multiple analytical steps.

This design does not preclude future orchestration. Automated pipelines, dashboards, agents, and workflow managers that compose **eventus** components can rely on each component behaving predictably and raising explicitly when its contract is violated. The bricks must be clear before the building can be automated.

Semantics

The semantics abstraction maps column names to concepts. In any real dataset, column names are idiosyncratic — one dataset calls the entity identifier `patient_id`, another calls it `member_id`. Without a semantics object, every downstream class may need to accept column names as constructor arguments; alternatively, the script-writer must remember what global constant names refer to what concept of the event. **eventus** solves this once. The analyst defines a semantics object at the top of the analysis, passes it into the constructor of a cleaner, and everything downstream uses it.

Two fields deserve particular attention. First, the field `also_defined_by`. It answers: beyond entity and date, what else defines whether two records represent the same episode? A patient may have overlapping hospitalizations at different hospitals (e.g., a transfer). These are different episodes. Setting `also_defined_by = ["hospital_id"]` tells the cleaner it may only merge intervals where `hospital_id` also matches. This is not a default assumption — it is a declared scientific decision, versioned in the semantics file.

Second, the field `descriptor_cols` declares columns that describe the episode but do not define its identity (for example, BMI, primary diagnosis, etc.). The `DescriptorColConfig` declares both the column type (category or numeric) and a timeline rule controlling how the column's values are carried into the `CohortTimeline` (sequence, unique, average, or none). This is a carriage decision — it governs what the `CohortTimeline` carries forward for downstream stratification. It is distinct from the aggregation rule in the `EpisodesCleanerConfig`, which governs how values are combined when overlapping episode records are merged during cleaning. The two decisions are independent and live in separate config files. Semantics objects can be built from YAML files or defined directly in code. Either way, the object is defined once.

Data objects

Data objects are validated, self-describing input containers. A **pandas** `DataFrame` is not designed to know what its columns mean or whether its data is clean; an **eventus** data object knows both. Every data object carries a semantics object alongside its data, binding the column declarations from Section 2.1 directly to the rows they govern. If a data object exists, it is complete: constructors raise on invalid inputs, producing no partial objects and no silent failures.

`Episodes` guarantees that every row has a valid entity identifier, a start date, an end date, and that causality holds — start precedes or equals end.

`Events` guarantees that every row has a valid entity identifier and a date. Duplicate detection, date parsing, and null checking are not the caller's responsibility — they happened before the object existed.

`ObsPeriodPerEntity` guarantees one row per entity with a valid observation window. Four named constructors cover the most common derivation strategies: from fixed calendar dates, from an age window, from episode boundaries, and from the first and last event dates in an `EventsPerEntity` object. The construction path is a first-class attribute that

downstream code can inspect — a pipeline can know whether the observation period was calendar-derived or age-derived without the caller having to pass that information separately. Crucially, the same downstream pipeline accepts an `ObsPeriodPerEntity` regardless of how it was constructed. The construction decision is recorded in the object; it does not leak into the pipeline that consumes it.

Cleaners and filters

Cleaners are the auditable bridge between raw `DataFrames` and validated data objects. `EpisodesCleaner` applies a configurable pipeline: date parsing, normalization, null rejection, date floor and ceiling enforcement, causality checking, duplicate removal, and interval merging. Every rejected row is recorded with an explicit reason. Every repaired row is recorded with a description of what changed. The causality check, for example, accepts either “reject” — discard the offending row — or “swap” — repair it and record the repair. These are scientific decisions that belong in a versioned config file, not on line 34 of a script. When cleaning is complete, `get_report()` returns a structured audit object documenting every transformation decision: row counts before and after each step, rejection reasons, and repair descriptions. The report is a first-class output, not a log.

`EventsCleaner` applies the same philosophy to event data, with an additional consolidation step for same-date same-identity records that carry different descriptor values. How those values are combined — collecting all diagnoses from two claims for the same ED visit, taking the median of repeated blood pressure readings, keeping the first recorded mobility status — is declared in the `DescriptorColConfig` for each column, connecting this decision back to the semantics layer described in Section 2.1.

Filters subset validated data objects by entity, date range, or descriptor value. They are chainable and non-mutating: a filter applied to an `Episodes` object returns a new `Episodes` object, with all structural guarantees intact. The pipeline that receives the filtered object does not know a filter was applied — it knows only that it has a valid `Episodes` object.

Analyzers

Analyzers compute quantities from validated data objects and produce typed intermediates. They do not clean, and they do not re-validate structural invariants that upstream objects already guarantee. By the time an analyzer receives its input, soundness is established — the analyzer’s only job is to compute.

The primary first-level analyzers each take a `CohortTimeline` as input and produce a typed intermediate. `CohortTimelineEpisodeAnalyzer` computes episode coverage statistics — active days, inactive days, and gap breakdowns — with tiered validated denominators. The denominator is non-trivial because not every entity has episodes, and not every gap is the same kind of absence: a gap before the first episode, a gap between episodes, and a gap after the last episode are scientifically distinct. `CohortTimelineEventAnalyzer` computes per-entity volume, timing, and shape statistics including burstiness and memory — properties of the inter-event gap distribution that characterize whether an entity’s events arrive in bursts or regularly, and whether bursts tend to follow bursts.

`EventEpisodeAnalyzer` computes nearest-neighbor gap statistics between one event identity and one episode identity, with explicit NaN semantics: an entity with no hospitalizations genuinely has no event-to-episode gap, and the result carries that honestly. `EpisodeDurationAnalyzer` operates directly on an `Episodes` object rather than a `CohortTimeline`, because duration is a property of individual episodes and does not require an observation period.

`EventCoOccurrenceAnalyzer` is also a first-level analyzer, taking a `CohortTimeline` and producing per-entity co-occurrence summaries across three analytical dimensions: presence (`compute_presence()`), gap timing (`compute_gaps()`), and directionality (`compute_directionality()`). Each method returns a typed intermediate — `EventCoOccurrencePresenceResult`, `EventCoOccurrenceGapSummary`, or `EventCoOccurrenceDirectionalitySummary` — that can be used directly or passed to a second-level analyzer.

`EventCoOccurrenceGapAnalyzer` and `EventCoOccurrenceDirectionalityAnalyzer` are second-level analyzers. They do not take a `CohortTimeline` — they take a validated per-entity summary produced by the first-level analyzer. The raw data has already been read and summarized; the second-level analyzer adds only the statistical layer: a permutation null, a KS test or Wilcoxon signed-rank test, and an effect size. The two tiers are independently composable.

Adding all three co-occurrence analyzers required writing three new classes that consumed existing validated objects. The `CohortTimeline`, the cleaners, and all prior analyzers required zero changes — a direct demonstration of the open/closed commitment described in Section 1.3.

Intermediates

Intermediates are validated result objects produced by analyzers. Each carries a meaningful `__repr__` that summarizes its contents — entity count, identity, key statistics — so that exploratory analysis in a notebook or console requires no additional inspection code. Intermediates are designed to be consumed directly by visualizers, which is what enforces the honest objects commitment from Section 1.3: a visualizer receives a typed intermediate and renders it — it does not recompute, re-validate, or reinterpret.

The central intermediate is `CohortTimeline` — a per-entity table assembling observation periods, episodes, and events into a single validated structure. One row per entity. It sits at a unique position in the pipeline: it is produced from validated data objects by the assembler, and it is progressively enriched by analyzers, each of which returns a new `CohortTimeline` instance rather than mutating the existing one. Descriptor columns declared in the semantics layer are carried forward, enabling condition-based stratification without additional joins. The per-entity constraint is not arbitrary — it is what allows any analyzer or visualizer to consume a `CohortTimeline` without knowing what other components produced it. When a new analytical result requires a different structure, a new intermediate class is introduced rather than relaxing this constraint.

The `EventResult` family carries per-entity event statistics: `EventResultVolume` (counts and prevalence), `EventResultTiming` (nth-event timing and recency), and `EventResultShape` (behavioral fingerprint including burstiness, memory, density, and center of mass). `EpisodeDurationResult` carries per-episode durations with optional descriptor columns for stratification. `EventEpisodeResult` carries nearest-neighbor gap statistics between one event identity and one episode identity. `EpisodeCoverageSummary` and `EpisodeActivityOverTime` carry cohort-level coverage summaries and per-timepoint activity statistics respectively. All carry explicit eligibility thresholds — NaN values are scientifically meaningful absences of signal, not missing data. An entity with fewer than two events genuinely has no mean inter-event gap; an entity with no hospitalizations genuinely has no event-to-episode gap. The intermediate carries that honestly.

The `EventCoOccurrence` family carries co-occurrence results at two levels, corresponding to the two-tier analyzer architecture described in Section 2.4. Per-entity summaries — `EventCoOccurrencePresenceResult`, `EventCoOccurrenceGapSummary`, `EventCoOccurrenceDirectionalitySummary` — carry one row per entity with validated denominators and explicit NaN semantics. Cohort-level test objects — `EventCoOccurrenceGapTest`, `EventCoOccurrenceDirectionalityTest` — carry the permutation null.

Visualizers

Visualizers consume one intermediate and one configuration object and produce one plot. They do not compute. The intermediate already guarantees structural soundness; the configuration already owns every visual choice. The separation is strict — a visualizer that receives an `EventCoOccurrenceGapTest` renders it, and nothing else.

Configuration objects are validated dataclasses that check all parameters at construction. If a configuration object exists, it is valid. Invalid configurations raise immediately with specific messages before any drawing begins. Passing `x_axis: calendar` to `StackedTimelinePlotter` when entities have different observation period lengths raises with the exact count of differing entities and the two valid alternatives. Passing a bin configuration with `min > max` raises at config construction, not halfway through rendering a figure. Passing a non-string legend label raises with the field name and the type that was received. The error is always at the source of the mistake, never at the point of rendering.

Plotters are organized by analytical domain — episode plotters, event plotters, co-occurrence plotters, and violin plotters each live in their own subpackage. Three examples illustrate the range. `StackedTimelinePlotter` is the most compositional: it accepts a `CohortTimeline` with any combination of episode and event layers, reads colors, markers, gap segment labels, and legend placement from `StackedTimelineConfig`, and renders a per-entity horizontal timeline with full period-of-interest segmentation.

`EventCoOccurrenceGapPlotter` illustrates the second-level consumer pattern: it takes an `EventCoOccurrenceGapTest` — not a `CohortTimeline` — and renders a two-panel KDE figure comparing observed gap distributions to the permutation null. The raw data was read once, summarized once, tested once; the plotter receives only the result.

`ArraysViolinPlotter` is the composable primitive: it receives a `{key: np.ndarray}` dict

and an `ArraysViolinConfig` and draws one violin per key. Higher-level plotters — `EpisodeDurationViolinPlotter`, `EpisodeCoverageViolinPlotter` — build the arrays from their respective intermediates and delegate all drawing to `ArraysViolinPlotter`. The drawing logic is tested once, in one place.

Vignettes

The vignette series demonstrates the **eventus** framework across various pipelines on simulated data covering insurance coverage periods, nursing facility stays, emergency department visits, inpatient hospitalizations, and event co-occurrence analysis. Ten chapters progress from basic episode cleaning through temporal co-occurrence and directionality testing. All data are synthetic, generated with a fixed random seed using `generate_vignette_data.py`. All code, configuration files, synthetic data, and without-**eventus** comparison scripts are available in the repository.

Chapter 1 — Cleaning hospitalization records

Scenario: What is a clean hospitalization record in US administrative claims data, and how many of the 11,500 raw records survive cleaning?

In US administrative claims data, a hospitalization file may not be one row per hospitalization. A single inpatient stay may generate multiple billing rows — one per billing day, one per diagnosis code, one per provider — because hospitals submit claims to insurers on a per-service or per-day basis rather than as a single episode record. Transfers produce overlapping intervals at different hospitals that are clinically distinct episodes, not duplicates. However, same-hospital overlaps within a short gap may represent one continuous stay. Missing identifiers, implausible dates, and causality violations (i.e., discharge date before admission date) add further complexity.

Two YAML files define the analysis: one declaring the semantics (what the columns mean and what defines a unique hospitalization), one declaring the cleaning rules (how to handle each problem). An `EpisodeSemantics` object and an `EpisodesCleanerConfig` are built from these files. The `EpisodesCleaner` accepts the raw `DataFrame` along with both objects and applies a fixed pipeline: null entity ID rejection, date parsing, floor/ceiling enforcement, causality checking, duplicate removal, and interval merging. The result is two outputs: an `Episodes` object, which guarantees that every row is structurally sound, and a cleaning report.

The `EpisodesCleaner` object produces the following report:

```
Cleaning report ————— Total input rows: 11,500 —————
————— Rejected: duplicate_row: 8,624 (75.0 null_start_date: 341 (3.0
null_entity_id: 115 (1.0 end_before_start_rejected: 54 (0.5 before_date_floor: 1 (0.0
after_date_ceiling: 1 (0.0 ————— Total rejected: 9,136 (79.4
Clean rows: 2,297 (20.0
```

Every rejected row is accounted for with an explicit reason. A separate audit DataFrame records every repaired row with a description of what has changed. The cleaning results are therefore available for sharing with collaborators and review, requiring no additional code. A bonus vignette applies the same pattern to ED visit data using EventsCleaner (3,390 clean records from 5,459 raw rows).

This vignette demonstrates that cleaners serve as a reproducible and auditable gate to validated data objects, configuration files serve as the methods section, and audit trails are automatic outputs requiring no additional code.

Chapter 2 — Descriptor aggregation in nursing facility data

Scenario: Your nursing facility data comes month-by-month even when the records belong to the same stay. If you want to collapse them into meaningful episodes of care, how should clinical measurements be aggregated?

A nursing facility assessment file may also not be one row per stay as the data may be saved in one-month intervals. Furthermore, each resident has monthly measurements — blood pressure, BMI, mobility status — one row per assessment. Collapsing the data into episodes requires aggregation and may require a different rule per column: BMI might need median, systolic BP might need mean, and mobility status is categorical — averaging it makes no sense. Nothing in a standard **pandas** groupby prevents accidentally applying mean to a categorical column. And a sequence of independent | assisted | dependent tells a clinical story of decline that a unique set assisted | dependent | independent destroys entirely. These are scientific decisions that belong in a versioned file, not buried in lambdas on line 89 of a script.

Two YAML files again define the analysis. The semantics file declares `also_defined_by = ["facility_id"]` (i.e., stays at different facilities are different episodes), and declares `descriptor_cols`, which specifies the type of each column (numeric or category). The cleaner config declares the aggregation rule per column. The EpisodesCleaner validates that mean is not applied to a category column at construction.

The resulting Episodes object carries the aggregated descriptor columns. A resident with multiple assessments in one stay produces one row:

```
resident_id facility_id systolic_bp bmi mobility_status NF0043 Facility_A 103.92 24.9
dependent | assisted | dependent
```

Changing `systolic_bp` from mean to median is one word, easy to find in the cleaner config file. Running both as a multiverse analysis — examining how conclusions vary across multiple reasonable aggregation choices — requires two config files and zero code changes.

This vignette demonstrates that aggregation rules can be declared and versioned in YAML configuration files, that type declarations in semantics prevent silent errors at construction, and that the `also_defined_by / descriptor_cols` distinction — what defines an episode and what describes it — is a first-class architectural decision rather than a coding convention.

Chapter 3 — Episode duration analysis

Scenario: You have cleaned the nursing facility data. How long were the nursing facility stays, and does stay length differ across facilities?

Your clinical collaborator asks: “How long were the stays?” You reach for **matplotlib**. Before you write a single line of plotting code, you need to answer a question that sounds trivial: “What exactly am I plotting, and will I be able to reproduce it?”

Bin width is a scientific choice. A 7-day bin aligns with clinical discharge patterns. A 30-day bin hides variation. A 1-day bin shows noise. That choice belongs in a methods section, not buried in `bins=range(0, 200, 7)` on line 34 of a script. Which percentile lines — 25th, 50th, 75th, 90th? Do you label them? The KDE bandwidth is a choice too — Scott’s rule and Silverman’s rule (Silverman 1986) produce different curves. When your collaborator asks for duration by facility, you need consistent bins, consistent colors, and consistent percentile lines across both facilities. A script hardcoded for the unstratified case has to be rewritten.

In the **eventus** design, `EpisodeDurationAnalyzer` accepts the `Episodes` object directly and produces an `EpisodeDurationResult` — one row per episode, one `duration_days` column.

```
EpisodeDurationResult: n_episodes : 227 n_entities : 200 mean_duration : 112.0 days
median_duration : 113.0 days
```

Every visual decision lives in a versioned YAML config file — bin width, percentile lines, KDE bandwidth, colors per facility. `build_arrays(stratify_by="facility_id")` prepares stratified arrays for `ArraysViolinPlotter`. Adding facility stratification requires one argument change. The config file is the complete, reproducible record of what was plotted and how. Figure 1 shows the KDE. Figure 2 shows the stratified violin.

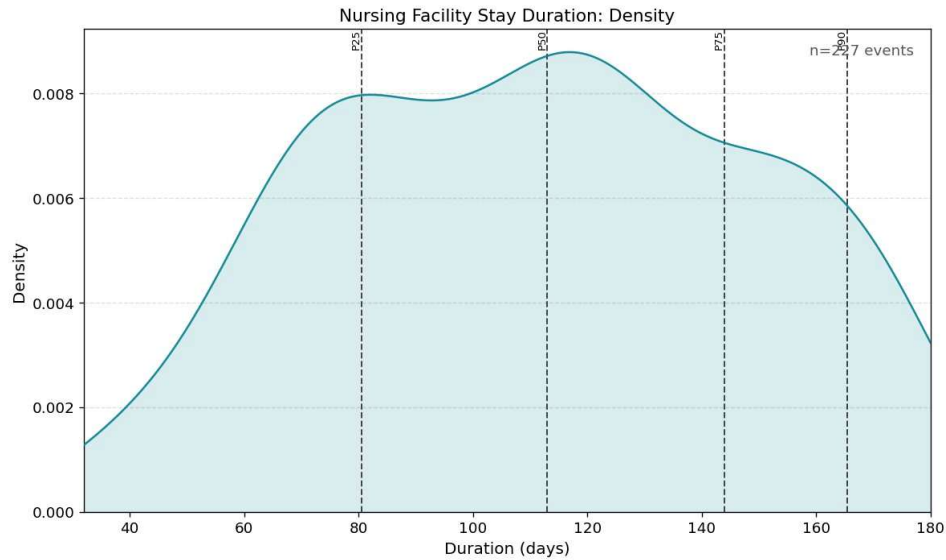


Figure 1. KDE of nursing facility stay duration ($n = 227$ episodes). Broad distribution peaking near 115 days with a secondary mode around 80 days. Dashed lines: P25, P50, P75, P90.

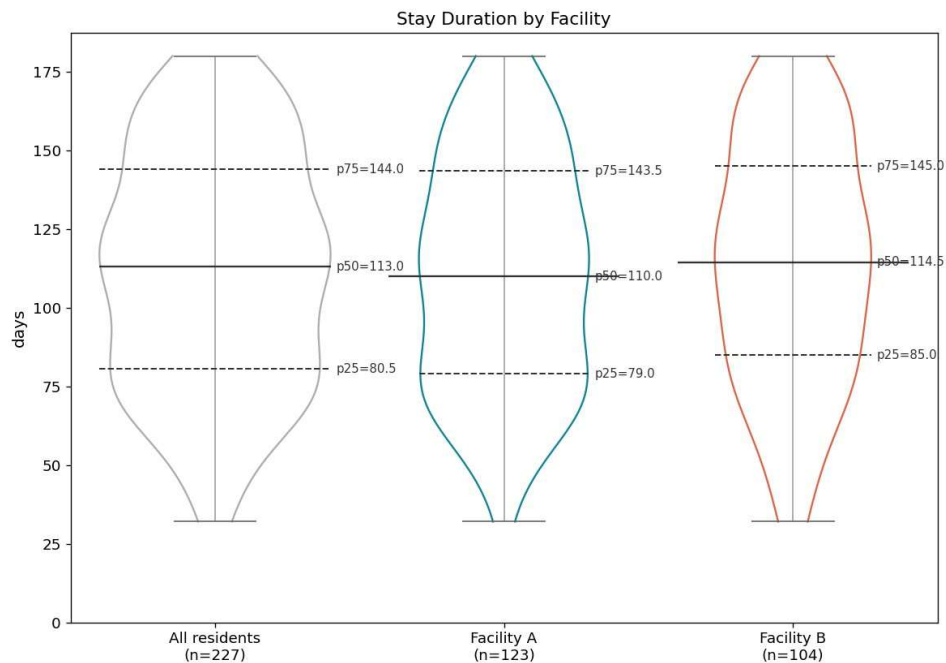


Figure 2. Stay duration stratified by facility. All residents ($n = 227$, $P50 = 113$ days), Facility A ($n = 123$, $P50 = 110$ days), Facility B ($n = 104$, $P50 = 114.5$ days).

This vignette demonstrates that visual and stratification decisions belong in versioned config files, not in plotting code — the config is the complete, reproducible record of what was plotted and how. Adding stratification requires one argument change in one config file.

Chapter 4 — Observation period construction

Scenario: How many days of Medicaid coverage did members have during calendar year 2022, and how does this change when the observation window is defined by age rather than calendar year?

You have coverage records for 500 Medicaid members. Before you can ask any scientific question — who was covered, for how long, were there gaps — you need to answer a structural one: *“What is each member’s observation window, and what happened inside it?”* This question has two versions. The first is simple: everyone shares the same calendar year. The second is harder and more realistic: each member has a different window based on their age.

Coverage data, or other episodes data, rarely align neatly with the period you are studying. This goes beyond simply filtering to a date range. The time before the first episode, between episodes, and after the last episode can matter depending on the analysis. For example, a member with no insurance coverage in the first quarter of their observation window represents a different situation from one who was covered from day one if certain interventions are meant to happen early in the observation period. Beyond clinical interpretation, this distinction also affects the correctness of any calculation that depends on a denominator. “19.4% of members had gaps” means 19.4% of members with any coverage, which is a different denominator than 19.4% of all members. Mixing them silently produces wrong statistics with no error and no warning.

Per-entity windows make this harder still. When the observation window is each member’s 18th to 25th birthday, every member has a different start and end date. Members who turned 25 before the study data begins have zero-length windows and must be handled explicitly rather than silently excluded. A script applies a date filter somewhere in the middle of the pipeline. It has no object. It has no name. It cannot be inspected or reused.

ObsPeriodPerEntity is a first-class object with a `construction_path` attribute recording exactly how it was built. Two paths are demonstrated: `construct_from_calendar()` gives every member the same 2022 window; `construct_from_age_window()` gives each member a personal 18–25 window from their date of birth. The rest of the pipeline is identical for both.

`EpisodesFilter(episodes).to_obs_period(obs, clip=True)` clips coverage periods to each member’s window. `CohortTimeline.build_from_components()` assembles the filtered episodes and observation period into one per-entity table, producing the first `CohortTimeline` in the vignette series:

```
CohortTimeline( entities : 495 entity_col : 'patient_id' has_obs_period : True
episode_identities : ['medicaid_coverage'] episode_descriptor_cols :
computed_episode_identities : [] event_identities : [] event_descriptor_cols :
computed_event_identities : [] )
```

`CohortTimelineEpisodeAnalyzer.enrich_with_episode_coverage()` enriches the timeline with seven `evt_comp_*` columns per member. These columns travel with the

object and are available for every downstream step without recomputing. A sample of the enriched data:

```
patient_id obs_start obs_end evt_medicaid_coverage_starts evt_comp..._active_days P0001
2022-01-01 2022-12-31 2022-01-01 | 2022-06-15 311 P0002 2022-01-01 2022-12-31
2022-03-10 296 P0003 2022-01-01 2022-12-31 NaN 0
```

P0001 had two coverage periods (pipe-delimited); P0002 had one; P0003 had none. `get_summary()` produces an `EpisodeCoverageSummary` with three tiers and validated denominators. Switching to the age window changes one constructor call. The age-window summary tells a different story: 31.1% of members had no coverage during ages 18–21, 29.8% were continuously covered for the full three years, and 24.3% had gaps in the middle of their coverage window.

This vignette demonstrates the value of treating the observation period as a first-class object rather than a date filter buried in the script. `CohortTimeline.build_from_components()` acts as a pure assembler: any valid `ObsPeriodPerEntity` and `Episodes` object snap together cleanly, because they have earned their structural guarantees upstream.

Chapter 5 — Stacked timeline visualization

Scenario: What do individual Medicaid coverage patterns look like across the 18–25 age window?

You know from Chapter 4 that 31.1% of members had no coverage during ages 18–21, that 29.8% were continuously covered for the full three years, and that 24.3% had gaps. A table of statistics tells you what happened. It cannot show you the pattern. Which members had gaps early in their window? Which had gaps late? A stacked timeline answers all of these at once — one bar per member, colored by coverage status.

No new cleaning. No new semantics. The `CohortTimeline` from Chapter 4 is the input. `ct.sample_subset(n=50, random_seed=42)` draws a reproducible 50-member sample — a display decision, not an analytical decision, explicitly separated from the analysis that used all 500 members. Every visual choice lives in a versioned YAML. Passing mode: `calendar` instead of passing mode: `auto` raised immediately, reporting the exact count of entities with differing observation periods. Figure 3 shows the result.

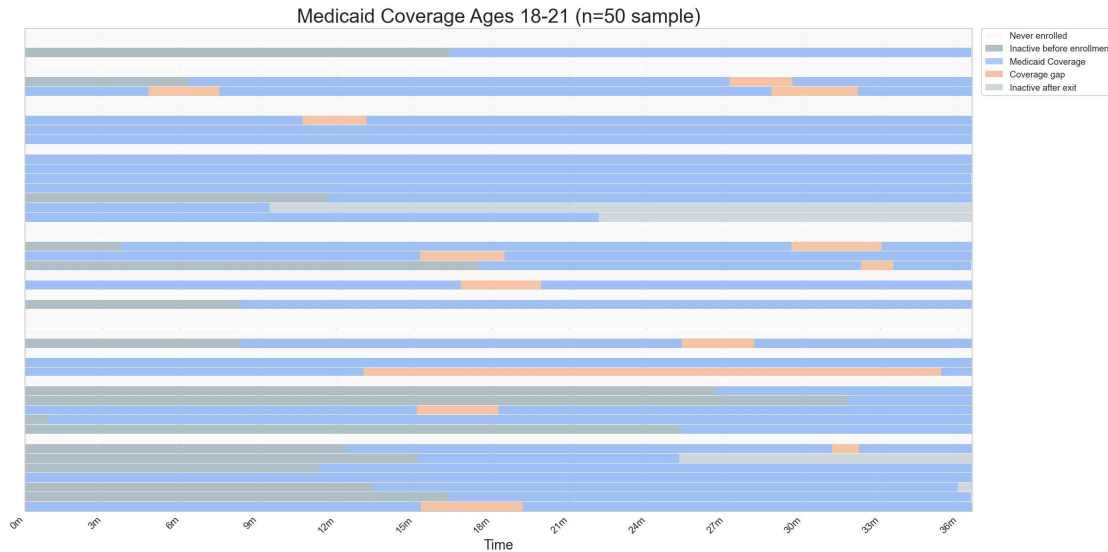


Figure 3. Stacked timeline of Medicaid coverage for 50 members, ages 18–21 (normalized x-axis, 0m–36m). Blue: active Medicaid coverage. Orange: coverage gap between episodes. Dark grey: inactive before first enrollment. Light grey: inactive after last exit. White: never enrolled. All five segment types are independently configured in the YAML and appear in the legend. A single member may show multiple segment types — the segments are distinct, not the members.

This vignette demonstrates how an intermediate object, `CohortTimeline`, serves as the contract between analysis and visualization. The plotter does not know how the observation periods were constructed, how the episodes were cleaned, or which entities were filtered; it receives a validated object and trusts it. The same plotter that draws calendar-year coverage draws age-window coverage with one config change and zero code changes. All visual decisions live in a versioned config file.

Chapter 6 — Event analysis

Scenario: How many ED visits did members have during calendar year 2022, and where in their coverage structure did those visits fall?

This chapter introduces two questions that require the same data but different analyzers. The first is a volume question: how many members had ED visits, and how many had multiple? The second is a cross-type question, only askable when both episodes and events live in the same validated structure: where in each member’s coverage structure did those visits fall — during active coverage, during a gap, before enrollment, or after exit?

Both questions begin with the same pipeline. Events need the same cleaning guarantees as episodes: null identifiers, unparseable dates, and duplicates handled explicitly. Events have one additional problem: same-date same-hospital records with different diagnoses are the same visit recorded twice, not two visits. `EventSemantics` declares `also_defined_by = ["hospital_id"]` — two visits on the same date at different hospitals are different events. The cleaner config declares consolidation rules: `icd10_condition: unique` and `systolic_bp: median`. These are scientific decisions that belong in a versioned config file.

`CohortTimeline.build_from_components()` assembles the coverage episodes and ED visits into one per-entity structure. `episode_identities` and `event_identities` are tracked separately — the timeline knows what it holds. The observation period is built from the coverage cohort — 793 members, calendar year 2022 — not from the event streams.

`CohortTimelineEventAnalyzer.compute_volume()` produces a validated result: 554 members (69.9%) had any ED visit in 2022, 325 (41.0%) had multiple. Figure 4 shows the prevalence bar. Figure 5 shows the count distribution — 30.1% had no visits, 28.9% had exactly one, 20.4% had two, with a long right tail extending to six or more. Switching to the 18–21 age window changes one constructor call: 514 (64.2%) with any visit, 382 (47.8%) with multiple. The higher repeat utilization rate reflects the longer observation window.

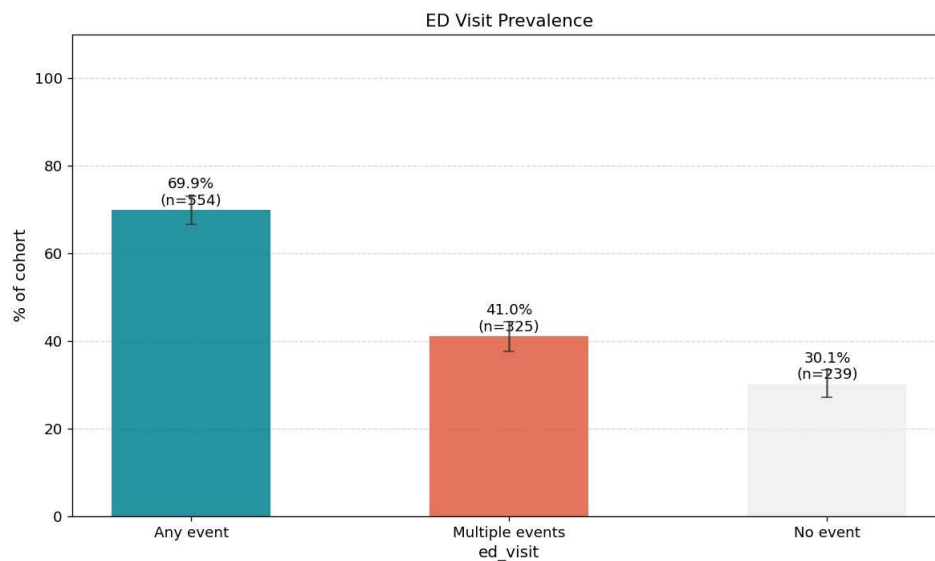


Figure 4. ED visit prevalence, calendar year 2022 ($n = 793$). 69.9% had any ED visit, 41.0% had multiple, 30.1% had none.

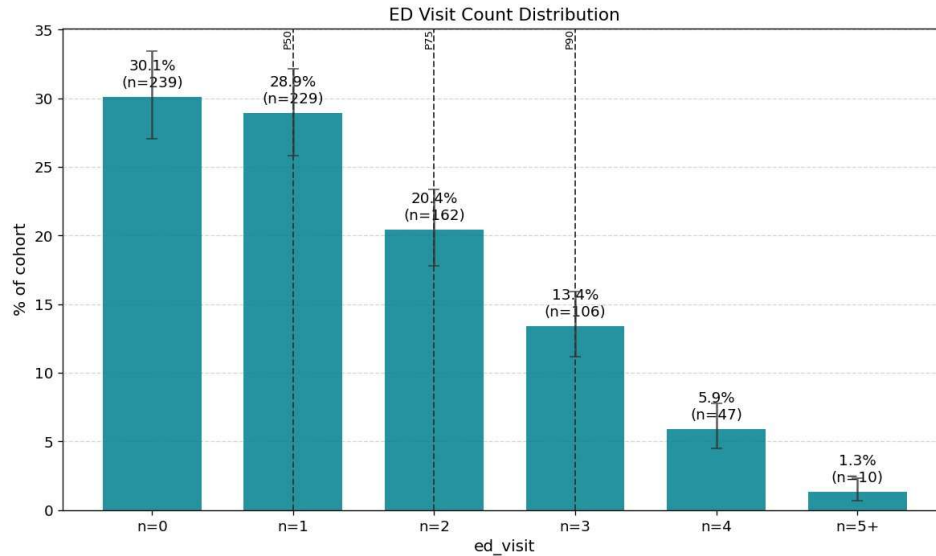


Figure 5. ED visit count distribution, calendar year 2022 ($n = 793$). Nearly a third had no visits. 28.9% had exactly one. 1.3% had five or more.

EpisodeEventInteractionAnalyzer then asks the cross-type question. The result:

EpisodeEventInteractionResult: episode_identity : medicaid_coverage event_identity :
 ed_visit entities : 793 events before first episode : 61 entities (7.7 events during active
 episodes: 509 entities (64.2 events during gaps : 19 entities (2.4 events after last episode :
 60 entities (7.6

2.4% of members visited the ED during a coverage gap. This question was not answerable from either data object alone. It required both, assembled in the same validated structure, with one analyzer call.

EpisodeEventInteractionAnalyzer is the fourth permutation of the pairwise analyzer design space, completing Events $\times$ ObsPeriod, Episodes $\times$ ObsPeriod, Events $\times$ Events, and Events $\times$ Episodes. Adding it required writing one new class consuming existing validated objects. The CohortTimeline, the cleaners, and all prior analyzers required zero changes.

Chapter 7 — Event timing and gap analysis

Scenario: When did members have their first, second, and third ED visit between ages 18 and 21, and does the gap between visits differ by diagnosis?

You know 64.2% of members in the null cohort and 67.6% in the signal cohort had at least one ED visit, and that more than half had multiple. Now the question is about when, and whether the pattern differs by condition. Two problems make this harder than it looks. Each n th timing has a different denominator: time to first visit is defined for 541 members (signal), time to second for 454, time to third for 354. A script must track three denominators manually and apply them correctly to three histograms on a shared axis. And stratifying gap distributions by condition requires that diagnosis labels survive from raw

claims through cleaning, consolidation, and timeline assembly — without any additional joins.

Chapter 7 reuses the full pipeline from Chapter 6. `compute_timing(max_n=3)` produces per-member timing with shrinking validated denominators: 541 (67.6%) with a first visit, 454 (56.8%) with a second, 354 (44.2%) with a third (signal cohort). Figure 6 shows faceted histograms on the same *x*-axis — each subplot shows its eligible denominator automatically. The first-visit distribution is left-skewed; the third-visit distribution is flatter and shifts right.

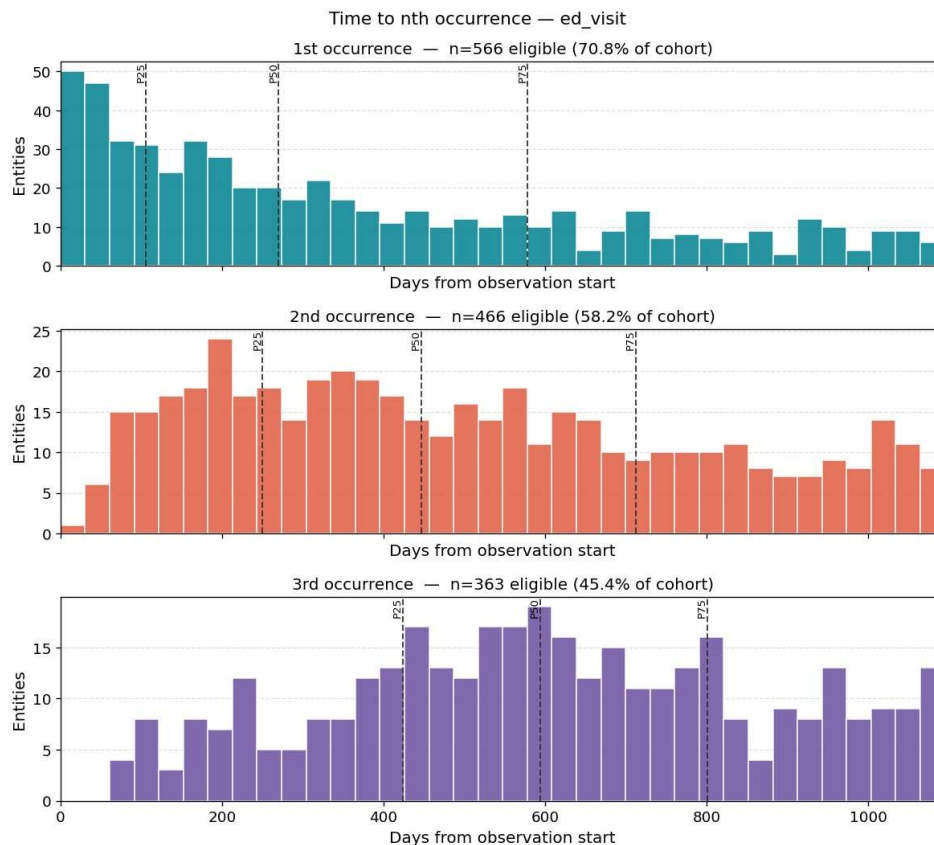


Figure 6. Time to *n*th ED visit from age 18, signal simulation (*n* = 800). Top: 1st visit (*n* = 541, 67.6%). Middle: 2nd (*n* = 454, 56.8%). Bottom: 3rd (*n* = 354, 44.2%). Shared *x*-axis. Dashed lines: P25, P50, P75.

`compute_shape()` produces per-entity behavioral fingerprints.

`plot_mean_gap_violin_stratified()` reads `evt_ed_visit_icd10_condition` directly from the CohortTimeline — declared in `EventSemantics.descriptor_cols` with `timeline: unique, no join required`. Figure 7 shows the signal: single-condition members (conditionA median 210 days, conditionB 184 days, conditionC 159 days) have longer gaps than comorbid members (149–178 days) — consistent with the higher visit rates assigned to conditionB and conditionC.

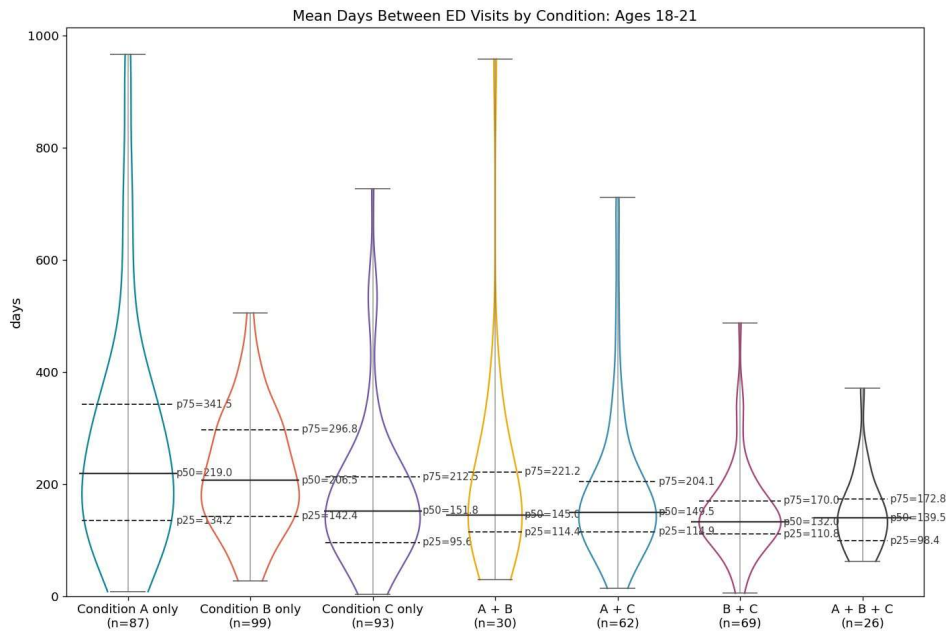


Figure 7. Mean days between ED visits by condition group, ages 18–21 (signal simulation). Single-condition members show longer gaps than comorbid members. Dashed lines: P25, P50, P75.

This vignette demonstrates how new timing and shape capabilities extend the framework without modifying existing components. Each carries its own validated denominators as properties, so the right denominator appears in each subplot title automatically rather than being tracked manually. Descriptor columns declared with `timeline: unique` survive the full pipeline into the `CohortTimeline` and are available for stratification without additional joins. Adding descriptor carriage required changes to `DescriptorColConfig`, `cohort_timeline_utils`, and `CohortTimeline`. Every analyzer, cleaner, and other intermediate required zero changes.

Chapter 8 — Event co-occurrence: Presence analysis

Scenario: Do cirrhosis diagnoses and ED visits co-occur in the same calendar year more often than chance would predict?

You have two cleaned event streams for a cohort of 5,000 patients: cirrhosis diagnosis dates and emergency department visits, both covering calendar year 2022. The scientific question is a presence question — not when events happened or which came first, but whether having a cirrhosis diagnosis is associated with a higher rate of ED visits in the same year.

Four structural problems make this harder than it looks in a script. First, co-occurrence counting requires the full patient list — including patients with neither event — as the denominator. A join that drops those patients corrupts all four cells of the 2×2 table. Second, the reference population must be the full declared cohort, not the union of the two event `DataFrames`. Third, a script produces ad hoc column names like `n_cirrh_and_ed` with

no documented meaning. Fourth, the Fisher exact test must be consistent with the same 2×2 table that produced the counts — a guarantee a script must engineer manually.

The observation period is constructed from the full 5,000-patient pool before either event stream is consulted. `CohortTimeline.build_from_components()` assembles both cleaned streams into one per-entity structure — all 5,000 patients present, including the 3,119 (62.4%) with neither event. They are the “neither” cell.

```
EventCoOccurrenceAnalyzer(ct, "cirrhosis_diagnosis",  
"ed_visit").compute_presence()
```

 produces:

```
EventCoOccurrencePresenceResult: n_with_a : 90 (1.8 n_with_b : 1,847 (36.9 n_with_both :  
56 (1.1 n_with_neither : 3,119 (62.4 p_b_given_a : 62.2 p_b_given_no_a : 36.5  
prevalence_ratio : 1.706x fisher_exact_p : 1.27e-06
```

Among cirrhosis patients, 62.2% had an ED visit in 2022 — nearly double the 36.5% rate among non-cirrhosis patients (prevalence ratio 1.71, 95% CI: 1.45–2.01, Fisher $p = 1.27 \times 10^{-6}$). With only 90 cirrhosis patients (1.8% prevalence), the confidence interval is wider than it would be at higher prevalence, but the signal is clear and consistent with the simulation design.

`presence.association` returns a full 2×2 association object with conditional probabilities, Wilson score confidence intervals, and a prevalence ratio with log-method CI. The disclaimer is built into the object because it is always true for observational co-occurrence statistics: these measures describe co-occurrence patterns within a defined observation period, not mechanisms or directionality.

The null simulation validates that the analyzer correctly identifies absence of signal. `simul_3` is two independent Poisson event streams with identical rates across all 5,000 patients — no planted relationship, no shared risk factors. The same pipeline, the same analyzer, the same result object: prevalence ratio 1.019, Fisher $p = 0.74$. Having event X provides no information about whether a patient also had event Y. The contrast with the signal result is the scientific validation: the analyzer detects signal when present and correctly returns null when not.

`EventCoOccurrenceAnalyzer` is a new class consuming an existing validated `CohortTimeline`. The cleaners, the timeline, and all prior analyzers required zero changes.

This vignette demonstrates that co-occurrence presence analysis is a first-class operation in **eventus**. Two independently cleaned event streams, assembled into one `CohortTimeline`, yield a validated 2×2 result with named properties, confidence intervals, and a built-in disclaimer — in three lines of analysis code. Presence analysis is the prerequisite for what follows: timing and directionality are only worth examining once a signal is established.

Chapter 9 — Event co-occurrence: Gap timing analysis

Scenario: Among patients who had both events, are they closer in time than independence would predict?

Chapter 8 established that cirrhosis diagnoses and ED visits co-occur above chance. Chapter 9 asks a different question: among patients who had both events, how close in time were they — and is that proximity shorter than what independence would predict? This is not a directionality question. Chapter 9 does not ask whether A came before B. It asks whether A and B cluster near each other in time, in either direction, more than chance alone would produce.

The primary effect size is the gap ratio: observed median gap divided by null median gap. EventCoOccurrenceGapAnalyzer constructs a patient-specific permutation null — both A and B dates are shuffled independently for each patient, preserving event counts and observation windows. A ratio of 0.45 means observed gaps are about half of what independence predicts. A ratio near 1.0 means no temporal signal.

compute_gaps() reuses the same CohortTimeline from Chapter 8. EventCoOccurrenceGapAnalyzer is a second-level analyzer — it takes an EventCoOccurrenceGapSummary as input, not a CohortTimeline. The gap summary is already validated. The analyzer adds the statistical layer without re-reading any raw data.

simul_4 (MI $\leftrightarrow$ stroke, symmetric ± 60 -day clustering, 385 co-occurring patients):

observed median (MI $\rightarrow$ stroke) : 26.0d null: 57.3d ratio: 0.454 ks_p: 5.80e-72
 observed median (stroke $\rightarrow$ MI) : 30.5d null: 68.1d ratio: 0.448 ks_p: 1.98e-64

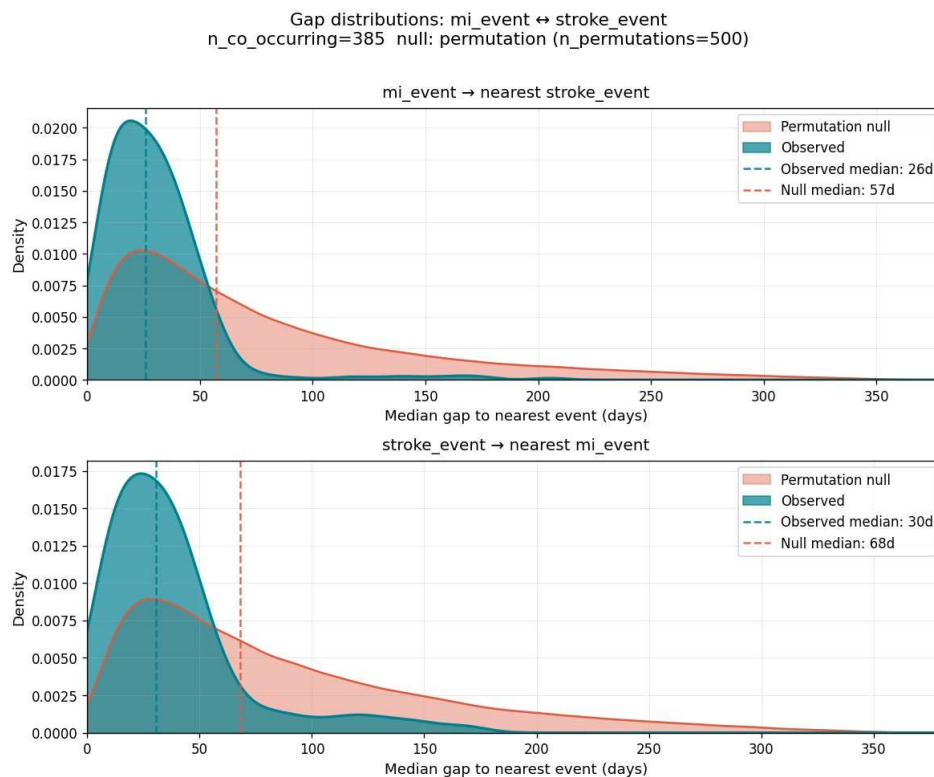


Figure 8. Gap distributions: $mi_event \leftrightarrow stroke_event$ (simul_4, $n = 385$ co-occurring).
 Teal: observed per-entity median gaps. Coral: permutation null. Top: MI $\rightarrow$ nearest stroke

(26d vs. 57d, ratio = 0.45). Bottom: stroke → nearest MI (30d vs. 68d, ratio = 0.45). Near-identical ratios confirm the symmetric undirected simulation design.

simul_3 (pure null, 335 co-occurring patients) produces gap ratio 1.019 (ks_p = 0.95) — observed and null distributions overlap nearly completely. The analyzer correctly identifies absence of temporal structure (Figure 9).

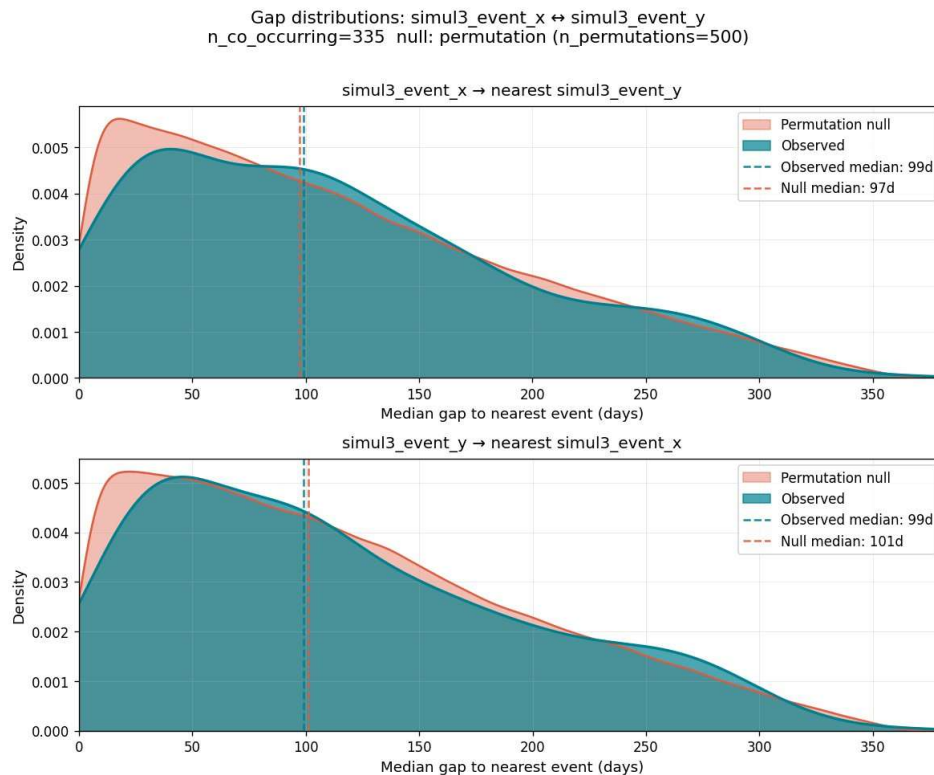


Figure 9. Gap distributions: *simul3_event_x* ↔ *simul3_event_y* (null, $n = 335$). Observed and null overlap (99d vs. 97d, ratio = 1.02). No temporal clustering.

simul_1 (cirrhosis → ED, directed signal, 56 co-occurring patients) reveals an asymmetry. The cirrhosis→ED ratio is 0.369 (29.5d vs. 79.9d). The ED→cirrhosis ratio is 0.466 (50.5d vs. 108.3d). The two directions are not symmetric: finding a cirrhosis diagnosis pulls closer to an ED visit than finding an ED visit pulls toward a cirrhosis diagnosis. This asymmetry is the first hint of directionality. Chapter 10 tests it directly.

EventCoOccurrenceGapAnalyzer and EventCoOccurrenceGapPlotter are new classes consuming existing validated objects. The CohortTimeline, cleaners, and all prior components required zero changes.

This vignette demonstrates that temporal clustering analysis is a natural extension of presence analysis — the same pipeline, one additional analyzer call. The gap ratio gives a magnitude the KS p -value alone cannot. Three scientifically distinct situations — undirected clustering, directed clustering, and pure null — are all correctly characterized by the same two method calls. The asymmetry in simul_1 is not a finding in Chapter 9; it is a question for Chapter 10.

Chapter 10 — Event co-occurrence: Directionality analysis

Scenario: Among patients who had both events and whose events cluster in time, does one tend to come before the other?

Chapters 8 and 9 established presence above chance and temporal clustering. Chapter 10 asks a distinct question: is the ordering consistent? Two event pairs can have identical gap ratios but completely different directionality patterns. A symmetric ± 60 -day clustering window produces the same gap ratio as a directed Exponential(30-day) window — but only one has a consistent ordering. Chapter 9 and Chapter 10 answer different scientific questions.

The mean signed gap is the aggregation strategy: for each A event, find the nearest B in either direction and record the signed gap (positive = B after A, negative = B before A), then take the mean across all A events for that patient. Mean aggregation captures directional tendency in ways that median absolute gaps cannot. The permutation null shuffles both A and B dates independently per patient, preserving event counts and observation windows. The null fraction A-first is computed empirically — not assumed to be 0.50. For patients with multiple events in a 365-day window, the null fraction is not exactly 0.50 in general.

`analyzer.compute_directionality()` returns an `EventCoOccurrenceDirectionalitySummary` with four explicit denominators: full cohort, patients with both events, patients with a non-NaN signed gap, and patients with unambiguous direction (excluding same-day ties). `EventCoOccurrenceDirectionalityAnalyzer` is a second-level analyzer — it takes the summary as input and adds the statistical layer without re-reading any raw data.

Results across three simulation groups:

```
simul_5 simul_4 simul_1 (resp->cardiac) (MI<->stroke) (cirrhosis->ED) n_with_both 868  
(17.4 fraction_a_first 62.1 direction_ratio 1.243 1.055 1.498 Wilcoxon p 1.62e-04 0.278  
(n.s.) 0.049
```

`simul_4` is the negative control: the symmetric ± 60 -day window produces `direction_ratio` 1.055 and Wilcoxon $p = 0.28$, confirming no consistent ordering. `simul_5` shows a clear directed signal (`direction_ratio` 1.243, $p = 1.62 \times 10^{-4}$). `simul_1` shows the strongest `direction_ratio` (1.498) — cirrhosis diagnosis precedes ED visits in 74.5% of co-occurring patients — despite only 56 co-occurring patients. The Wilcoxon $p = 0.049$ is weaker than `simul_5` not because the effect is weaker but because $n = 56$ vs. $n = 868$. Effect size and statistical power answer different questions. Figures 10–12 show the three cases.

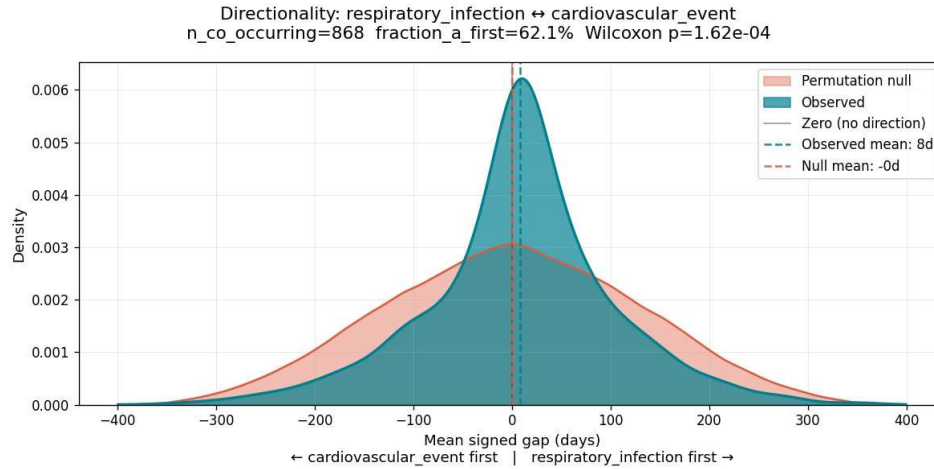


Figure 10. Directionality: respiratory_infection ↔ cardiovascular_event (simul_5, $n = 868$). Teal: observed per-patient mean signed gaps (positive = respiratory infection first). Coral: permutation null. The observed distribution is right-shifted (observed mean +8d, null -0d), fraction_a_first = 62.1% vs. null 50.0% (direction_ratio = 1.24, $p = 1.62 \times 10^{-4}$).

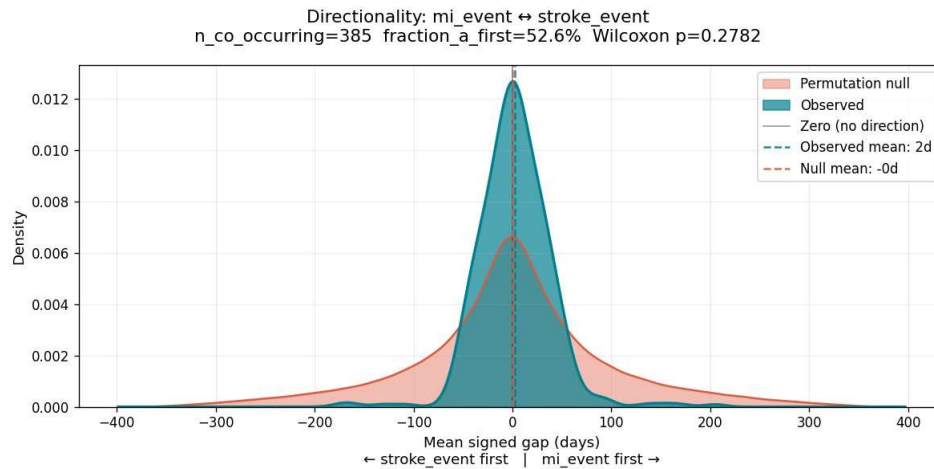


Figure 11. Directionality: mi_event ↔ stroke_event (simul_4 negative control, $n = 385$). Observed and null distributions overlap (fraction_a_first = 52.6%, direction_ratio = 1.055, $p = 0.28$). No consistent ordering detected, as expected from the symmetric ± 60 -day clustering design.

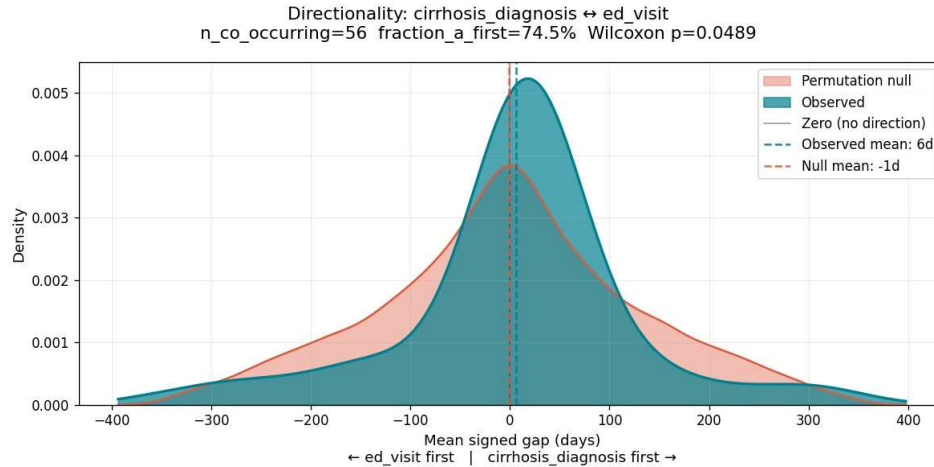


Figure 12. Directionality: *cirrhosis_diagnosis* ↔ *ed_visit* (*simul_1*, $n = 56$). Observed distribution right-shifted (mean +6d vs. null -1d), *fraction_a_first* = 74.5%, *direction_ratio* = 1.498, $p = 0.049$. Strongest *direction_ratio* of the three simulations; weakest p -value due to $n = 56$. Illustrates the distinction between effect size and statistical power.

`EventCoOccurrenceDirectionalityAnalyzer` and `EventCoOccurrenceDirectionalityPlotter` are new classes consuming existing validated objects. The `CohortTimeline`, cleaners, and all prior components across Chapters 8 and 9 required zero changes.

This vignette demonstrates that directionality is a distinct analytical dimension from temporal clustering. Chapters 8–10 form a complete co-occurrence pipeline: presence establishes the signal, gap timing establishes temporal proximity, and directionality establishes ordering. Each chapter adds one analytical dimension using the same event streams and the same pipeline structure. No chapter required changes to any previous chapter’s code.

Comparison with script-based implementations

To evaluate the practical cost of the script-based paradigm, we implemented equivalent pipelines for six vignettes using standard **pandas**, **NumPy**, and **matplotlib** (McKinney 2010; Harris et al. 2020; Hunter 2007). Scripts were generated by a large language model given identical specifications and refactored to use global constants. Table 1 summarizes the results.

*Comparison of **eventus** and script-based implementations.*

Vignette	Script lines	eventus lines	Script issues
Ch 1 — Episode cleaning	~150	~10	No audit trail; no versioned config; no per-row guarantees
Ch 1B — Event cleaning	~70	~10	No audit trail; no versioned config; no per-row guarantees
Ch 2 — Descriptor aggregation	~150	~10	Aggregation rules hardcoded
Ch 4 — Observation periods	~275	~20	Runs with pandas UserWarning
Ch 5 — Stacked timeline	~165	~3	Normalized x -axis only
Ch 6 — Event volume	~99	~15	Age window not implemented

Line counts alone understate the difference. Four structural limitations appear consistently across all script implementations regardless of code quality. First, no versioned config: cleaning and aggregation parameters live as hardcoded values invisible to collaborators reading the output. Second, no per-row audit trail: scripts produce aggregate counts; **eventus** produces a rejected DataFrame and a modified DataFrame, both accessible programmatically. Third, silent errors instead of specific exceptions: the Chapter 4 script produced a **pandas** UserWarning and continued execution; the Chapter 5 script silently produced a normalized x -axis when calendar mode was requested for variable-length observation periods. Fourth, adaptation requires rewriting: switching from calendar to age window analysis required ~40 additional lines in the script and one constructor call change in **eventus**.

Chapters 3 and 7 were not implemented as script comparisons. Chapter 3 was estimated at ~140 lines. Chapter 7 was estimated at ~240 lines before accounting for the per-entity age window (~75 additional lines) and condition stratification join (~30 lines). The escalating complexity — from ~150 lines in Chapter 1 to ~275 lines in Chapter 4 to not implemented in Chapters 3 and 7 — is itself the argument.

Additional capabilities

The ten vignettes do not cover all analytical capabilities in the current release.

Event-episode temporal analysis. EventEpisodeAnalyzer computes nearest-neighbor gap statistics between one event identity and one episode identity within a CohortTimeline. For each entity, it records: events within any episode interval (count and percentage), nearest event-to-episode-start gap in days, and nearest episode-discharge-to-event gap in days. NaN values are scientifically meaningful absences of signal — entities with no hospitalizations have NaN for all episode-related statistics, not missing data. The analyzer has no config — no thresholds, no windows — because the computation is fully determined by the data. This capability was demonstrated with a cohort of 775 members

having both ED visits and inpatient hospitalizations, producing 16.5% of ED visits during or on the day of a hospitalization (billing artifacts and ED-triggered admissions). The open/closed principle holds: adding `EventEpisodeAnalyzer` required writing one new class consuming existing validated objects; the assembler, the timeline, and all existing analyzers required zero changes.

Episode activity over time.

`CohortTimelineEpisodeAnalyzer.compute_activity_over_time()` produces a per-timepoint activity timeseries in normalized (relative to each entity's `obs_start`) or calendar mode. The result feeds `ActivityOverTimePlotter` with a two-panel output: coverage curve and entry/exit flow.

Episode coverage summary. `get_summary()` produces an `EpisodeCoverageSummary` with three tiers and validated denominators, preventing the silent denominator errors that script-based summaries are prone to.

Event behavioral fingerprint. `compute_shape()` produces burstiness, memory, density, and center of mass per entity. `plot_fingerprint()` renders a burstiness vs. memory scatter with four behavioral quadrants.

Planned extensions. Descriptor-based filtering (selection operation σ in relational algebra; (Codd 1970)) and interactive visualization are planned for future releases. Event-event co-occurrence analysis is implemented in the current release through `EventCoOccurrenceAnalyzer` (Chapters 8–10).

Discussion

Implications for reproducibility in health services research

The reproducibility crisis in computational science has been extensively documented (Stodden, Seiler, and Ma 2018; Sandve et al. 2013; Wilson et al. 2017). Health services research is particularly vulnerable: analyses involve complex multi-step pipelines, large administrative datasets with known quality issues, and analytical decisions that are rarely reported in published methods sections. **eventus** addresses this structurally. It does not ask analysts to document their decisions more carefully. It makes the decisions impossible to make without documenting them. The config file is not optional documentation — it is the required input to the cleaner. The audit trail is not an optional report — it is an automatic output of every cleaning run.

Domain agnosticism and generalizability

The three-object structure — entities, observation periods, interval episodes, point-in-time events — recurs across domains wherever entities are observed over time: insurance analytics, educational longitudinal research, ecological monitoring (Michener and Jones 2012), clinical trial follow-up, and workforce analysis. Engineering applications offer a particularly direct mapping: in structural health monitoring, the observation period is the monitoring window, interval episodes are periods of sensor-recorded loading or stress, and

point-in-time events are inspections or detected defect occurrences. Cleaners remove invalid sensor readings; analyzers compute damage evolution metrics such as crack growth rates or fatigue cycle counts. New analyzers implementing domain-specific methods — rainflow counting for fatigue analysis, crack-growth models — can be added within the same framework without modifying any existing component. **eventus** is designed to be applicable to any of these domains without modification. We hope this work is useful to researchers in other domains who face the same structural challenge and to software developers designing similar frameworks.

Object-oriented design in applied data science

The object-oriented design patterns used in **eventus** — typed objects with validated invariants, layered architecture with explicit boundaries, configuration objects as first-class citizens — are standard in software engineering but underutilized in applied data science. **eventus** demonstrates that these patterns are not only applicable to data analysis but produce measurable benefits: shorter calling code, automatic audit trails, reproducible visualization, and feature additions that propagate with minimal surface area. When descriptor columns were added to the `CohortTimeline`, the change required modifications to exactly four components; every other component in the pipeline required zero changes.

Compatibility with large language model-assisted development

A practical consideration for any scientific software framework in the current era is its compatibility with large language model (LLM) assisted analysis. Increasingly, analysts write code with the assistance of LLMs. A framework built on clearly defined typed objects, versioned configuration files, and comprehensive documentation provides the structural scaffolding that may make LLM-assisted analysis more reliable. The module `README` becomes a specification that an LLM can learn. The type system becomes a contract that an LLM can respect. The **eventus** design is such that if an LLM is asked to extend an **eventus** pipeline, it can produce consistent, architecturally correct code — rather than writing bespoke **pandas** scripts from scratch, which tend to produce variable and often subtly incorrect results. Object-oriented frameworks with clear semantics are not in tension with LLM-assisted development. They are likely to make it better and more efficient.

The general principle that clearer specifications and structured interfaces improve LLM code generation is well-documented (Chen et al. 2021; Austin et al. 2021). Whether OOP frameworks specifically outperform script-based approaches for LLM-assisted scientific analysis is an open empirical question beyond the scope of this paper. We raise the point as a reasonable inference and direction worth investigating.

Limitations and future directions

The pipe-delimited storage format in `CohortTimeline` is compact and human-readable but requires parsing at every analytical step; for very large cohorts this may introduce performance costs that a sparse matrix representation would avoid. Python’s interpreted execution adds further overhead relative to compiled alternatives for computationally intensive operations. However, the layered architecture makes targeted optimization

tractable. Because each layer has a single responsibility (Single Responsibility Principle; (Martin 2000)) and depends on typed object interfaces rather than concrete implementations (Dependency Inversion Principle), any individual layer can be reimplemented in C, C++, or Rust without modifying any other layer. A C extension that produces a valid `CohortTimeline` object is, by the Liskov Substitution Principle (Liskov and Wing 1994), interchangeable with the pure Python implementation from the perspective of every downstream component. The typed object boundaries that enforce architectural discipline also make performance optimization safe — the contract between layers is the interface, not the implementation.

The co-occurrence analysis layer supports event-event analysis through `EventCoOccurrenceAnalyzer` (presence, gap timing, directionality) with permutation-based statistical testing. Event-episode analysis is available through `EventEpisodeAnalyzer`. Episode-episode analyzers and sequential dependency analysis (Markov chain characterization of event type ordering) are planned for a future release. The visualization layer currently produces static plots; interactive visualization is a direction for future development.

Conclusion

eventus provides a unified, object-oriented framework for longitudinal cohort analysis in which observation periods, interval episodes, and point-in-time events are first-class typed objects. The six-layer architecture carries guarantees forward from raw data to publication-ready visualization, making reproducibility a structural property of the pipeline. The ten vignettes demonstrate that the framework produces correct, auditable, reproducible analyses with substantially less code than equivalent script-based implementations, and that script-based implementations cannot provide the same structural guarantees regardless of code quality. The framework is domain-agnostic, LLM-compatible, and designed to be extended.

References

Alam, Sajid, Nok Lam Chan, Gabriel Comym, Yetunde Dada, Ivan Danov, Deepyaman Datta, Tynan DeBold, et al. 2024. *Kedro*. <https://github.com/kedro-org/kedro>.

Austin, Jacob, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, et al. 2021. “Program Synthesis with Large Language Models.” <https://arxiv.org/abs/2108.07732>.

Bantilan, Niels. 2020. “Pandera: Statistical Data Validation of Pandas Dataframes.” In *Proceedings of the 19th Python in Science Conference*, edited by Meghann Agarwal, Chris Calloway, Dillon Niederhut, and David Shupe, 116–24. <https://doi.org/10.25080/Majora-342d178e-010>.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. 2021. “Evaluating Large Language Models Trained on Code.” <https://arxiv.org/abs/2107.03374>.

- Codd, Edgar F. 1970. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM* 13 (6): 377–87. <https://doi.org/10.1145/362384.362685>.
- Dima, Alexandra L., and Dan Dediu. 2017. "Computation of Adherence to Medication and Visualization of Medication Histories in r with AdhereR: Towards Transparent and Reproducible Use of Electronic Healthcare Data." *PLOS ONE* 12 (4): e0174426. <https://doi.org/10.1371/journal.pone.0174426>.
- Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Great Expectations Team. 2024. *Great Expectations: Always Know What to Expect from Your Data*. https://github.com/great-expectations/great_expectations.
- Grolemund, Garrett, and Hadley Wickham. 2011. "Dates and Times Made Easy with Lubridate." *Journal of Statistical Software* 40 (3): 1–25. <https://doi.org/10.18637/jss.v040.i03>.
- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. <https://doi.org/10.1038/s41586-020-2649-2>.
- Hemkens, Lars G., Eric I. Benchimol, Sinead M. Langan, Matthias Briel, Benjamin Kasenda, Januario Januel, Eric Herrett, Erik von Elm, Vivian Nguyen, and Liam Smeeth. 2016. "The Reporting of Studies Using Routinely Collected Health Data Was Often Insufficient." *Journal of Clinical Epidemiology* 79: 104–11. <https://doi.org/10.1016/j.jclinepi.2016.06.005>.
- Hripcsak, George, Jon D. Duke, Nigam H. Shah, Christian G. Reich, Vojtech Huser, Martijn J. Schuemie, Marc A. Suchard, et al. 2015. "Observational Health Data Sciences and Informatics (OHDSI): Opportunities for Observational Researchers." *Studies in Health Technology and Informatics* 216: 574–78.
- Hunter, John D. 2007. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering* 9 (3): 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Iannone, Richard, and Mauricio Vargas. 2024. *Pointblank: Data Validation and Organization of Metadata for Local and Remote Tables*. <https://CRAN.R-project.org/package=pointblank>.
- Köster, Johannes, and Sven Rahmann. 2012. "Snakemake — a Scalable Bioinformatics Workflow Engine." *Bioinformatics* 28 (19): 2520–22. <https://doi.org/10.1093/bioinformatics/bts480>.
- Landau, William Michael. 2021. *Targets: Dynamic Function-Oriented Make-Like Declarative Pipelines for r*. <https://doi.org/10.21105/joss.02959>.
- Lavallee, Martin, and Adam Black. 2024. *Capr: Cohort Definition Application Programming*. <https://ohdsi.github.io/Capr/>.

- Liskov, Barbara, and Jeannette Wing. 1994. "A Behavioral Notion of Subtyping." *ACM Transactions on Programming Languages and Systems* 16 (6): 1811–41. <https://doi.org/10.1145/197320.197383>.
- Loo, Mark P. J. van der, and Edwin de Jonge. 2021. "Data Validation Infrastructure for r." *Journal of Statistical Software* 97 (10): 1–31. <https://doi.org/10.18637/jss.v097.i10>.
- Martin, Robert C. 2000. "Design Principles and Design Patterns." *Object Mentor* 1 (34): 597.
- McKinney, Wes. 2010. "Data Structures for Statistical Computing in Python." In *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- Michener, William K., and Matthew B. Jones. 2012. "Ecoinformatics: Supporting Ecology as a Data-Intensive Science." *Trends in Ecology & Evolution* 27 (2): 85–93. <https://doi.org/10.1016/j.tree.2011.11.016>.
- Nab, Linda, Andrea L. Schaffer, William J. Hulme, Nicholas J. DeVito, Iain Dillingham, Milan Wiedemann, Colm D. Andrews, et al. 2024. "OpenSAFELY: A Platform for Analysing Electronic Health Records Designed for Reproducible Research." *Pharmacoepidemiology and Drug Safety* 33 (6): e5815. <https://doi.org/10.1002/pds.5815>.
- Patel, Chirag J., Belinda Burford, and John P. A. Ioannidis. 2015. "Assessment of Vibration of Effects Due to Model Specification Can Demonstrate the Instability of Observational Associations." *Journal of Clinical Epidemiology* 68 (9): 1046–58. <https://doi.org/10.1016/j.jclinepi.2015.05.029>.
- Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27. <https://doi.org/10.1126/science.1213847>.
- Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. "Ten Simple Rules for Reproducible Computational Research." *PLOS Computational Biology* 9 (10): e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>.
- Silverman, Bernard W. 1986. *Density Estimation for Statistics and Data Analysis*. London: Chapman & Hall.
- Simonsohn, Uri, Joseph P. Simmons, and Leif D. Nelson. 2020. "Specification Curve Analysis." *Nature Human Behaviour* 4: 1208–14. <https://doi.org/10.1038/s41562-020-0912-z>.
- Steege, Sara, Francis Tuerlinckx, Andrew Gelman, and Wolf Vanpaemel. 2016. "Increasing Transparency Through a Multiverse Analysis." *Perspectives on Psychological Science* 11 (5): 702–12. <https://doi.org/10.1177/1745691616658637>.
- Stodden, Victoria, Jennifer Seiler, and Zhaokun Ma. 2018. "An Empirical Analysis of Journal Policy Effectiveness for Computational Reproducibility." *Proceedings of the National Academy of Sciences* 115 (11): 2584–89. <https://doi.org/10.1073/pnas.1708290115>.

Therneau, Terry M. 2023. *Survival: A Package for Survival Analysis in r*. <https://CRAN.R-project.org/package=survival>.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan. 2023. *Dplyr: A Grammar of Data Manipulation*. <https://CRAN.R-project.org/package=dplyr>.

Wilson, Greg, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. 2017. "Good Enough Practices in Scientific Computing." *PLOS Computational Biology* 13 (6): e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>.